

API Security under OWASP API Security Top 10

	Details
Institute	Institute of Business Administration, Karachi
School	School of Mathematics and Computer Science
Department	Computer Science
Program	Master of Science in Computer Science (MSCS)
Semester	Spring 2024
Student	Muhammad Faizan Siddiqui – 27228

Table of Contents

API Security under OWASP API Security Top 10

- Table of Contents
- Ranking of the API Security Issues based on the Criticality
- Examples of Common Vulnerabilities for each API Security Issues
 - API1:2023 - Broken Object Level Authorization
 - Ranking Reason
 - Explanation
 - Examples
 - Example 1: Inadequate User Validation
 - Vulnerable Code
 - Secure Code
 - Example 2: Lack of Ownership Verification
 - Vulnerable Code
 - Secure Code
 - Example 3: Missing Role-Based Access Control
 - Vulnerable Code
 - Secure Code
 - API2:2023 - Broken Authentication
 - Ranking Reason
 - Explanation
 - Examples
 - Example 1: Hardcoded Credentials
 - Vulnerable Code
 - Secure Code
 - Example 2: Insecure Session Management
 - Vulnerable Code
 - Secure Code
 - Example 3: Missing Rate Limiting
 - Vulnerable Code

Secure Code

API5:2023 - Broken Function Level Authorization

Ranking Reason

Explanation

Examples

Example 1: No Authorization Check

Vulnerable Code

Secure Code

Example 2: Insecure Direct Object References

Vulnerable Code

Secure Code

Example 3: Missing Role Check in URL Routing

Vulnerable Code

Secure Code

API6:2023 - Unrestricted Access to Sensitive Business Flows

Ranking Reason

Explanation

Examples

Example 1: Sensitive Financial Operations Accessible Without Authentication

Vulnerable Code

Secure Code

Example 2: Sensitive Data Export Function Accessible Without Authorization

Vulnerable Code

Secure Code

Example 3: Administrative Operations Accessible to Regular Users

Vulnerable Code

Secure Code

API7:2023 - Server Side Request Forgery

Ranking Reason

Explanation

Examples

Example 1: Basic SSRF Vulnerability

Vulnerable Code

Secure Code

Example 2: SSRF with Redirects

Vulnerable Code

Secure Code

Example 3: SSRF with POST Requests

Vulnerable Code

Secure Code

API4:2023 - Unrestricted Resource Consumption

Ranking Reason

Explanation

Examples

Example 1: Unrestricted Loop in a Route

Vulnerable Code

Secure Code

Example 2: Unrestricted File Upload

Vulnerable Code

Secure Code

Example 3: Unrestricted Database Queries

Vulnerable Code

Secure Code

API8:2023 - Security Misconfiguration

Ranking Reason

Explanation

Examples

Example 1: Debug Mode Enabled in Production

Vulnerable Code

Secure Code

Example 2: Using Default Secret Key

Vulnerable Code

Secure Code

Example 3: Insecure HTTP Headers

Vulnerable Code

Secure Code

API3:2023 - Broken Object Property Level Authorization

Ranking Reason

Explanation

Examples

Example 1: Lack of Authorization Check

Vulnerable Code

Secure Code

Example 2: Insecure Direct Object Reference (IDOR)

Vulnerable Code

Secure Code

Example 3: Improper Use of Object References

Vulnerable Code

Secure Code

API10:2023 - Unsafe Consumption of APIs

Ranking Reason

Explanation

Examples

Example 1: Lack of Input Validation

Vulnerable Code

Secure Code

Example 2: Insufficient Authentication Check

Vulnerable Code

Secure Code

Example 3: Insecure Data Transmission

Vulnerable Code

Secure Code

API9:2023 - Improper Inventory Management

Ranking Reason

Explanation

Examples

Example 1: Lack of Input Validation

Vulnerable Code

Secure Code

Example 2: Insecure Direct Object Reference (IDOR)

Vulnerable Code

Secure Code

Example 3: Insufficient Authentication and Authorization

Vulnerable Code

Secure Code

Conclusion

Ranking of the API Security Issues based on the Criticality

Ranking API security issues based on their criticality involves considering factors such as potential impact, ease of exploitation, and the prevalence of the vulnerability in real-world scenarios. Here is a proposed ranking of the provided API security issues from most to least critical.

Ranking	API Security Issue	Proposed Percentage of Criticality
1	API1:2023 - Broken Object Level Authorization	95%
2	API2:2023 - Broken Authentication	90%
3	API5:2023 - Broken Function Level Authorization	85%
4	API6:2023 - Unrestricted Access to Sensitive Business Flows	80%
5	API7:2023 - Server Side Request Forgery	75%
6	API4:2023 - Unrestricted Resource Consumption	70%
7	API8:2023 - Security Misconfiguration	65%
8	API3:2023 - Broken Object Property Level Authorization	60%
9	API10:2023 - Unsafe Consumption of APIs	55%
10	API9:2023 - Improper Inventory Management	50%

This ranking considers the potential for exploitation, the impact on confidentiality, integrity, and availability, as well as the likelihood of these vulnerabilities being exploited in typical API environments.

Examples of Common Vulnerabilities for each API Security Issues

API1:2023 - Broken Object Level Authorization

Ranking Reason

This vulnerability allows attackers to access objects they are not authorized to, leading to data breaches and unauthorized data manipulation.

Explanation

Broken Object Level Authorization occurs when an application does not properly enforce access controls, allowing unauthorized users to access or manipulate objects (such as database records) they should not have access to.

Examples

Example 1: Inadequate User Validation

In this example, an API endpoint allows users to access details of any user by specifying the user ID in the URL. The endpoint fails to check whether the requesting user is authorized to access the requested user's details.

Vulnerable Code

```
from flask import Flask, request, jsonify

app = Flask(__name__)

# Mock user data
users = {
    1: {"name": "Alice", "email": "alice@example.com"},
    2: {"name": "Bob", "email": "bob@example.com"}
}

@app.route('/user/<int:user_id>', methods=['GET'])
def get_user(user_id):
    user = users.get(user_id)
    if user:
        return jsonify(user)
    else:
        return jsonify({"error": "User not found"}), 404

if __name__ == '__main__':
    app.run(debug=True)
```

Secure Code

```
from flask import Flask, request, jsonify

app = Flask(__name__)
```

```

# Mock user data
users = {
    1: {"name": "Alice", "email": "alice@example.com"},
    2: {"name": "Bob", "email": "bob@example.com"}
}

# Mock API keys for users
api_keys = {
    "key_for_alice": 1,
    "key_for_bob": 2
}

def get_user_id_from_api_key(api_key):
    return api_keys.get(api_key)

@app.route('/user/<int:user_id>', methods=['GET'])
def get_user(user_id):
    api_key = request.headers.get('X-API-Key')
    if not api_key:
        return jsonify({"error": "Missing API key"}), 401

    requesting_user_id = get_user_id_from_api_key(api_key)
    if not requesting_user_id:
        return jsonify({"error": "Invalid API key"}), 403

    if requesting_user_id != user_id:
        return jsonify({"error": "Unauthorized access"}), 403

    user = users.get(user_id)
    if user:
        return jsonify(user)
    else:
        return jsonify({"error": "User not found"}), 404

if __name__ == '__main__':
    app.run(debug=True)

```

Example 2: Lack of Ownership Verification

In this example, a user can update the details of any other user by specifying the user ID in the request payload. The endpoint does not verify if the authenticated user has permission to update the target user's details.

Vulnerable Code

```

from flask import Flask, request, jsonify

app = Flask(__name__)

# Mock user data
users = {
    1: {"name": "Alice", "email": "alice@example.com"},
    2: {"name": "Bob", "email": "bob@example.com"}
}

@app.route('/user/<int:user_id>', methods=['PUT'])

```

```

def update_user(user_id):
    if user_id in users:
        user_data = request.json
        users[user_id].update(user_data)
        return jsonify({"success": "User updated successfully"})
    else:
        return jsonify({"error": "User not found"}), 404

if __name__ == '__main__':
    app.run(debug=True)

```

Secure Code

```

from flask import Flask, request, jsonify

app = Flask(__name__)

# Mock user data
users = {
    1: {"name": "Alice", "email": "alice@example.com"},
    2: {"name": "Bob", "email": "bob@example.com"}
}

# Mock token data
tokens = {
    "token123": 1, # Token for user 1
    "token456": 2 # Token for user 2
}

def get_authenticated_user():
    token = request.headers.get('Authorization')
    if token and token in tokens:
        return tokens[token]
    return None

@app.route('/user/<int:user_id>', methods=['PUT'])
def update_user(user_id):
    authenticated_user_id = get_authenticated_user()
    if authenticated_user_id is None:
        return jsonify({"error": "Unauthorized"}), 401

    if authenticated_user_id != user_id:
        return jsonify({"error": "Forbidden"}), 403

    if user_id in users:
        user_data = request.json
        users[user_id].update(user_data)
        return jsonify({"success": "User updated successfully"})
    else:
        return jsonify({"error": "User not found"}), 404

if __name__ == '__main__':
    app.run(debug=True)

```


Example 3: Missing Role-Based Access Control

In this example, an API endpoint allows users to delete any user's account by specifying the user ID. There is no check to ensure that only users with the appropriate role (e.g., admin) can perform the delete operation.

Vulnerable Code

```
from flask import Flask, request, jsonify

app = Flask(__name__)

# Mock user data
users = {
    1: {"name": "Alice", "email": "alice@example.com"},
    2: {"name": "Bob", "email": "bob@example.com"}
}

@app.route('/user/<int:user_id>', methods=['DELETE'])
def delete_user(user_id):
    if user_id in users:
        del users[user_id]
        return jsonify({"success": "User deleted successfully"})
    else:
        return jsonify({"error": "User not found"}), 404

if __name__ == '__main__':
    app.run(debug=True)
```

Secure Code

```
from flask import Flask, request, jsonify

app = Flask(__name__)

# Mock user data
users = {
    1: {"name": "Alice", "email": "alice@example.com"},
    2: {"name": "Bob", "email": "bob@example.com"}
}

# Mock admin check function
def is_admin():
    # For demonstration, assume user is determined via request header
    # In real applications, use proper authentication mechanisms
    auth_header = request.headers.get('Authorization')
    return auth_header == 'Bearer admin-token'

@app.route('/user/<int:user_id>', methods=['DELETE'])
def delete_user(user_id):
    if not is_admin():
        return jsonify({"error": "Unauthorized"}), 403

    if user_id in users:
        del users[user_id]
        return jsonify({"success": "User deleted successfully"})
    else:
        return jsonify({"error": "User not found"}), 404
```

```
if __name__ == '__main__':  
    app.run(debug=True)
```

API2:2023 - Broken Authentication

Ranking Reason

Weak authentication mechanisms can allow attackers to gain unauthorized access to systems, compromising data integrity and confidentiality.

Explanation

Broken Authentication occurs when the authentication mechanisms are poorly implemented, allowing attackers to compromise user accounts.

Examples

Example 1: Hardcoded Credentials

This example demonstrates the use of hardcoded credentials, which can easily be discovered and exploited.

Vulnerable Code

```
from flask import Flask, request, jsonify  
  
app = Flask(__name__)  
  
# Hardcoded credentials  
USERNAME = 'admin'  
PASSWORD = 'password123'  
  
@app.route('/login', methods=['POST'])  
def login():  
    data = request.get_json()  
    if data['username'] == USERNAME and data['password'] == PASSWORD:  
        return jsonify({"message": "Login successful!"}), 200  
    else:  
        return jsonify({"message": "Invalid credentials"}), 401  
  
if __name__ == '__main__':  
    app.run(debug=True)
```

Secure Code

```
from flask import Flask, request, jsonify  
import os  
from dotenv import load_dotenv  
  
# Load environment variables from .env file  
load_dotenv()  
  
app = Flask(__name__)  
  
# Get credentials from environment variables  
USERNAME = os.getenv('USERNAME')  
PASSWORD = os.getenv('PASSWORD')
```

```
@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    if data['username'] == USERNAME and data['password'] == PASSWORD:
        return jsonify({"message": "Login successful!"}), 200
    else:
        return jsonify({"message": "Invalid credentials"}), 401

if __name__ == '__main__':
    app.run(debug=True)
```

Example 2: Insecure Session Management

This example demonstrates insecure session management where the session ID is predictable.

Vulnerable Code

```
from flask import Flask, request, jsonify, session
import hashlib

app = Flask(__name__)
app.secret_key = 'supersecretkey'

# User data for demonstration purposes
users = {
    'user1': 'password1',
    'user2': 'password2'
}

@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    username = data['username']
    password = data['password']

    if username in users and users[username] == password:
        # Predictable session ID
        session_id = hashlib.md5(username.encode()).hexdigest()
        session['user'] = session_id
        return jsonify({"message": "Login successful!", "session_id": session_id}), 200
    else:
        return jsonify({"message": "Invalid credentials"}), 401

if __name__ == '__main__':
    app.run(debug=True)
```

Secure Code

```
from flask import Flask, request, jsonify, session

app = Flask(__name__)
app.secret_key = 'supersecretkey' # Ensure this is kept secret and secure

# User data for demonstration purposes
users = {
    'user1': 'password1',
```

```

    'user2': 'password2'
}

@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    username = data.get('username')
    password = data.get('password')

    if username in users and users[username] == password:
        session['user'] = username
        return jsonify({"message": "Login successful!"}), 200
    else:
        return jsonify({"message": "Invalid credentials!"}), 401

@app.route('/logout', methods=['POST'])
def logout():
    session.pop('user', None)
    return jsonify({"message": "Logout successful!"}), 200

if __name__ == '__main__':
    app.run(debug=True)

```

Example 3: Missing Rate Limiting

This example demonstrates missing rate limiting, allowing attackers to brute-force login attempts.

Vulnerable Code

```

from flask import Flask, request, jsonify

app = Flask(__name__)

# User data for demonstration purposes
users = {
    'user1': 'password1',
    'user2': 'password2'
}

@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()
    username = data['username']
    password = data['password']

    if username in users and users[username] == password:
        return jsonify({"message": "Login successful!"}), 200
    else:
        return jsonify({"message": "Invalid credentials!"}), 401

if __name__ == '__main__':
    app.run(debug=True)

```

```

from flask import Flask, request, jsonify
from flask_limiter import Limiter
from flask_limiter.util import get_remote_address

app = Flask(__name__)

# Initialize the rate limiter
limiter = Limiter(
    key_func=get_remote_address,
    default_limits=["5 per minute"]
)

# Attach the limiter to the app
limiter.init_app(app)

# User data for demonstration purposes
users = {
    'user1': 'password1',
    'user2': 'password2'
}

@app.route('/login', methods=['POST'])
@limiter.limit("5 per minute") # Limit to 5 login attempts per minute per IP
def login():
    data = request.get_json()
    username = data['username']
    password = data['password']

    if username in users and users[username] == password:
        return jsonify({"message": "Login successful!"}), 200
    else:
        return jsonify({"message": "Invalid credentials"}), 401

if __name__ == '__main__':
    app.run(debug=True)

```

API5:2023 - Broken Function Level Authorization

Ranking Reason

This can allow attackers to access functions they should not be able to, leading to privilege escalation and unauthorized actions.

Explanation

Broken Function Level Authorization occurs when an application fails to properly enforce access control on a function or action, allowing unauthorized users to perform actions they shouldn't be able to.

Examples

Example 1: No Authorization Check

In this example, a sensitive admin function is accessible to any authenticated user without checking their role or permissions.

Vulnerable Code

```
from flask import Flask, request, jsonify, session

app = Flask(__name__)
app.secret_key = 'supersecretkey'

# Simulating a user database
users = {
    'user1': {'username': 'user1', 'password': 'password1', 'role': 'user'},
    'admin': {'username': 'admin', 'password': 'adminpass', 'role': 'admin'}
}

# Simulated login route
@app.route('/login', methods=['POST'])
def login():
    username = request.json.get('username')
    password = request.json.get('password')
    user = users.get(username)
    if user and user['password'] == password:
        session['username'] = username
        return jsonify({'message': 'Logged in successfully!'})
    return jsonify({'message': 'Invalid credentials!'}), 401

# Vulnerable admin function without role check
@app.route('/admin/delete_user', methods=['POST'])
def delete_user():
    if 'username' not in session:
        return jsonify({'message': 'Unauthorized!'}), 401

    user_to_delete = request.json.get('username')
    if user_to_delete in users:
        del users[user_to_delete]
        return jsonify({'message': f'User {user_to_delete} deleted successfully!'})
    return jsonify({'message': 'User not found!'}), 404

if __name__ == '__main__':
    app.run(debug=True)
```

Secure Code

```
from flask import Flask, request, jsonify, session

app = Flask(__name__)
app.secret_key = 'supersecretkey'

# Simulating a user database
users = {
    'user1': {'username': 'user1', 'password': 'password1', 'role': 'user'},
    'admin': {'username': 'admin', 'password': 'adminpass', 'role': 'admin'}
```

```

}

# Simulated login route
@app.route('/login', methods=['POST'])
def login():
    username = request.json.get('username')
    password = request.json.get('password')
    user = users.get(username)
    if user and user['password'] == password:
        session['username'] = username
        session['role'] = user['role']
        return jsonify({'message': 'Logged in successfully!'})
    return jsonify({'message': 'Invalid credentials!'}), 401

# Admin function with role check
@app.route('/admin/delete_user', methods=['POST'])
def delete_user():
    if 'username' not in session or session.get('role') != 'admin':
        return jsonify({'message': 'Unauthorized!'}), 401

    user_to_delete = request.json.get('username')
    if user_to_delete in users:
        del users[user_to_delete]
        return jsonify({'message': f'User {user_to_delete} deleted successfully!'})
    return jsonify({'message': 'User not found!'}), 404

if __name__ == '__main__':
    app.run(debug=True)

```

Example 2: Insecure Direct Object References

In this example, a user can manipulate the request to access an endpoint that should be restricted to administrators.

Vulnerable Code

```

from flask import Flask, request, jsonify, session

app = Flask(__name__)
app.secret_key = 'supersecretkey'

# Simulating a user database
users = {
    'user1': {'username': 'user1', 'password': 'password1', 'role': 'user'},
    'admin': {'username': 'admin', 'password': 'adminpass', 'role': 'admin'}
}

# Simulated login route
@app.route('/login', methods=['POST'])
def login():
    username = request.json.get('username')
    password = request.json.get('password')
    user = users.get(username)
    if user and user['password'] == password:
        session['username'] = username
        session['role'] = user['role']

```

```

        return jsonify({'message': 'Logged in successfully!'})
    return jsonify({'message': 'Invalid credentials!'}), 401

# Vulnerable endpoint accessible by role manipulation
@app.route('/admin/get_all_users', methods=['GET'])
def get_all_users():
    role = request.args.get('role')
    if role == 'admin':
        return jsonify({'users': list(users.keys())})
    return jsonify({'message': 'Unauthorized!'}), 401

if __name__ == '__main__':
    app.run(debug=True)

```

Secure Code

```

from flask import Flask, request, jsonify, session

app = Flask(__name__)
app.secret_key = 'supersecretkey'

# Simulating a user database
users = {
    'user1': {'username': 'user1', 'password': 'password1', 'role': 'user'},
    'admin': {'username': 'admin', 'password': 'adminpass', 'role': 'admin'}
}

# Simulated login route
@app.route('/login', methods=['POST'])
def login():
    username = request.json.get('username')
    password = request.json.get('password')
    user = users.get(username)
    if user and user['password'] == password:
        session['username'] = username
        session['role'] = user['role']
        return jsonify({'message': 'Logged in successfully!'})
    return jsonify({'message': 'Invalid credentials!'}), 401

# Secure endpoint restricted to administrators based on session role
@app.route('/admin/get_all_users', methods=['GET'])
def get_all_users():
    # Check if the user is logged in and has the 'admin' role
    if session.get('role') == 'admin':
        return jsonify({'users': list(users.keys())})
    return jsonify({'message': 'Unauthorized!'}), 401

if __name__ == '__main__':
    app.run(debug=True)

```

Example 3: Missing Role Check in URL Routing

In this example, the URL routing is structured in a way that does not enforce role checks, allowing unauthorized access to certain endpoints.

Vulnerable Code

```
from flask import Flask, request, jsonify, session

app = Flask(__name__)
app.secret_key = 'supersecretkey'

# Simulating a user database
users = {
    'user1': {'username': 'user1', 'password': 'password1', 'role': 'user'},
    'admin': {'username': 'admin', 'password': 'adminpass', 'role': 'admin'}
}

# Simulated login route
@app.route('/login', methods=['POST'])
def login():
    username = request.json.get('username')
    password = request.json.get('password')
    user = users.get(username)
    if user and user['password'] == password:
        session['username'] = username
        session['role'] = user['role']
        return jsonify({'message': 'Logged in successfully!'})
    return jsonify({'message': 'Invalid credentials!'}), 401

# No role check on the endpoint
@app.route('/admin/update_user/<username>', methods=['POST'])
def update_user(username):
    if 'username' not in session:
        return jsonify({'message': 'Unauthorized!'}), 401

    new_data = request.json
    if username in users:
        users[username].update(new_data)
        return jsonify({'message': f'User {username} updated successfully!'})
    return jsonify({'message': 'User not found!'}), 404

if __name__ == '__main__':
    app.run(debug=True)
```

Secure Code

```
from flask import Flask, request, jsonify, session

app = Flask(__name__)
app.secret_key = 'supersecretkey'

# Simulating a user database
users = {
    'user1': {'username': 'user1', 'password': 'password1', 'role': 'user'},
    'admin': {'username': 'admin', 'password': 'adminpass', 'role': 'admin'}
}

# Simulated login route
@app.route('/login', methods=['POST'])
def login():
```

```

username = request.json.get('username')
password = request.json.get('password')
user = users.get(username)
if user and user['password'] == password:
    session['username'] = username
    session['role'] = user['role']
    return jsonify({'message': 'Logged in successfully!'})
return jsonify({'message': 'Invalid credentials!'}), 401

# Role check added to the endpoint
@app.route('/admin/update_user/<username>', methods=['POST'])
def update_user(username):
    if 'username' not in session:
        return jsonify({'message': 'Unauthorized!'}), 401

    if session.get('role') != 'admin':
        return jsonify({'message': 'Forbidden! You do not have the required role.'}),
403

    new_data = request.json
    if username in users:
        users[username].update(new_data)
        return jsonify({'message': f'User {username} updated successfully!'})
    return jsonify({'message': 'User not found!'}), 404

if __name__ == '__main__':
    app.run(debug=True)

```

API6:2023 - Unrestricted Access to Sensitive Business Flows

Ranking Reason

Exposing sensitive business processes can lead to significant operational disruption and financial loss.

Explanation

Unrestricted Access to Sensitive Business Flows occurs when sensitive business processes are exposed without proper access controls.

Examples

Example 1: Sensitive Financial Operations Accessible Without Authentication

In this example, a financial transaction endpoint is exposed without any authentication, allowing unauthorized users to perform sensitive operations.

Vulnerable Code

```

from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/transfer', methods=['POST'])
def transfer():
    from_account = request.json.get('from_account')
    to_account = request.json.get('to_account')
    amount = request.json.get('amount')

```

```

# Simulate a transfer operation
return jsonify({
    'status': 'success',
    'message': f'Transferred {amount} from {from_account} to {to_account}'
}), 200

if __name__ == '__main__':
    app.run(debug=True)

```

Secure Code

```

from flask import Flask, request, jsonify, abort

app = Flask(__name__)

# Simulated token for authentication
AUTH_TOKEN = 'your_secret_token_here'

def authenticate():
    token = request.headers.get('Authorization')
    if token != f'Bearer {AUTH_TOKEN}':
        abort(403) # Forbidden

@app.route('/transfer', methods=['POST'])
def transfer():
    authenticate()

    data = request.json
    from_account = data.get('from_account')
    to_account = data.get('to_account')
    amount = data.get('amount')

    # Basic validation
    if not from_account or not to_account or not amount:
        return jsonify({
            'status': 'error',
            'message': 'Missing required fields'
        }), 400

    try:
        amount = float(amount) # Ensure amount is a number
    except ValueError:
        return jsonify({
            'status': 'error',
            'message': 'Invalid amount'
        }), 400

    # Simulate a transfer operation
    return jsonify({
        'status': 'success',
        'message': f'Transferred {amount} from {from_account} to {to_account}'
    }), 200

if __name__ == '__main__':
    app.run(debug=True)

```

Example 2: Sensitive Data Export Function Accessible Without Authorization

In this example, a sensitive data export endpoint is accessible to all users without proper authorization checks.

Vulnerable Code

```
from flask import Flask, send_file
import csv

app = Flask(__name__)

@app.route('/export-data', methods=['GET'])
def export_data():
    # Simulate creating a CSV file with sensitive data
    with open('sensitive_data.csv', mode='w') as file:
        writer = csv.writer(file)
        writer.writerow(['ID', 'Name', 'Sensitive Info'])
        writer.writerow([1, 'John Doe', 'Secret1'])
        writer.writerow([2, 'Jane Smith', 'Secret2'])

    return send_file('sensitive_data.csv', as_attachment=True)

if __name__ == '__main__':
    app.run(debug=True)
```

Secure Code

```
from flask import Flask, send_file, request, Response
import csv
import functools

app = Flask(__name__)

# Dictionary to store username and password
users = {
    'admin': 'password123'
}

def check_auth(username, password):
    """Check if a username/password combination is valid."""
    return username in users and users[username] == password

def authenticate():
    """Send a 401 response to enable basic auth"""
    return Response(
        'Could not verify your access level for that URL.\n'
        'You have to login with proper credentials', 401,
        {'WWW-Authenticate': 'Basic realm="Login Required"'})

def requires_auth(f):
    """Decorator to prompt for basic auth"""
    @functools.wraps(f)
    def decorated(*args, **kwargs):
        auth = request.authorization
        if not auth or not check_auth(auth.username, auth.password):
            return authenticate()
    return decorated
```

```

        return f(*args, **kwargs)
    return decorated

@app.route('/export-data', methods=['GET'])
@requires_auth
def export_data():
    # Simulate creating a CSV file with sensitive data
    with open('sensitive_data.csv', mode='w', newline='') as file:
        writer = csv.writer(file)
        writer.writerow(['ID', 'Name', 'Sensitive Info'])
        writer.writerow([1, 'John Doe', 'Secret1'])
        writer.writerow([2, 'Jane Smith', 'Secret2'])

    return send_file('sensitive_data.csv', as_attachment=True)

if __name__ == '__main__':
    app.run(debug=True)

```

Example 3: Administrative Operations Accessible to Regular Users

In this example, an administrative operation endpoint (such as deleting users) is accessible without checking if the user has admin privileges.

Vulnerable Code

```

from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/delete-user', methods=['POST'])
def delete_user():
    user_id = request.json.get('user_id')

    # Simulate deleting a user
    return jsonify({
        'status': 'success',
        'message': f'User with ID {user_id} deleted'
    }), 200

if __name__ == '__main__':
    app.run(debug=True)

```

Secure Code

```

from flask import Flask, request, jsonify, abort

app = Flask(__name__)

# Simulated admin token
ADMIN_TOKEN = 'secret_admin_token'

def check_admin_privileges():
    token = request.headers.get('Authorization')
    if token != ADMIN_TOKEN:
        abort(403, description="Access denied: Admin privileges required")

@app.route('/delete-user', methods=['POST'])

```

```
def delete_user():
    check_admin_privileges() # Check if the user has admin privileges

    user_id = request.json.get('user_id')
    if not user_id:
        return jsonify({'status': 'error', 'message': 'User ID is required'}), 400

    # Simulate deleting a user
    return jsonify({
        'status': 'success',
        'message': f'User with ID {user_id} deleted'
    }), 200

if __name__ == '__main__':
    app.run(debug=True)
```

API7:2023 - Server Side Request Forgery

Ranking Reason

SSRF can be used to access internal systems and data, potentially leading to critical data breaches and system compromise.

Explanation

SSRF occurs when an attacker can make the server perform requests to unintended locations, potentially accessing internal services.

Examples

Example 1: Basic SSRF Vulnerability

This example demonstrates a basic SSRF vulnerability where the server accepts a URL as a parameter and fetches its content without any validation.

Vulnerable Code

```
from flask import Flask, request, jsonify
import requests

app = Flask(__name__)

@app.route('/fetch')
def fetch():
    url = request.args.get('url')
    if not url:
        return jsonify({'error': 'No URL provided'}), 400
    try:
        response = requests.get(url)
        return jsonify({'content': response.text})
    except Exception as e:
        return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    app.run(debug=True)
```

Secure Code

```
from flask import Flask, request, jsonify
import requests
from urllib.parse import urlparse

app = Flask(__name__)

# List of allowed domains
ALLOWED_DOMAINS = ['example.com', 'another-allowed-domain.com']

def is_url_allowed(url):
    try:
        parsed_url = urlparse(url)
        domain = parsed_url.netloc
        return any(domain.endswith(allowed_domain) for allowed_domain in
ALLOWED_DOMAINS)
    except Exception:
        return False

@app.route('/fetch')
def fetch():
    url = request.args.get('url')
    if not url:
        return jsonify({'error': 'No URL provided'}), 400

    if not is_url_allowed(url):
        return jsonify({'error': 'URL is not allowed'}), 400

    try:
        response = requests.get(url)
        return jsonify({'content': response.text})
    except Exception as e:
        return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    app.run(debug=True)
```

Example 2: SSRF with Redirects

This example shows an SSRF vulnerability where the server fetches a URL that may redirect to another potentially malicious URL.

Vulnerable Code

```
from flask import Flask, request, jsonify
import requests

app = Flask(__name__)

@app.route('/redirect')
def redirect_fetch():
    url = request.args.get('url')
    if not url:
        return jsonify({'error': 'No URL provided'}), 400
    try:
        response = requests.get(url, allow_redirects=True)
```

```

        return jsonify({'content': response.text})
    except Exception as e:
        return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    app.run(debug=True)

```

Secure Code

```

from flask import Flask, request, jsonify
import requests
from urllib.parse import urlparse

app = Flask(__name__)

# Define a list of allowed domains
ALLOWED_DOMAINS = {'example.com', 'anotherexample.com'}

def is_url_safe(url):
    try:
        parsed_url = urlparse(url)
        domain = parsed_url.netloc
        return domain in ALLOWED_DOMAINS
    except Exception:
        return False

@app.route('/redirect')
def redirect_fetch():
    url = request.args.get('url')
    if not url:
        return jsonify({'error': 'No URL provided'}), 400

    if not is_url_safe(url):
        return jsonify({'error': 'URL is not allowed'}), 400

    try:
        response = requests.get(url, allow_redirects=True, timeout=5)
        return jsonify({'content': response.text})
    except Exception as e:
        return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    app.run(debug=True)

```

Example 3: SSRF with POST Requests

This example demonstrates an SSRF vulnerability where the server accepts a URL and POST data, and sends a POST request to the URL.

Vulnerable Code

```

from flask import Flask, request, jsonify
import requests

app = Flask(__name__)

@app.route('/post')

```



```

def post_fetch():
    url = request.args.get('url')
    data = request.args.get('data', {})
    if not url:
        return jsonify({'error': 'No URL provided'}), 400
    try:
        response = requests.post(url, data=data)
        return jsonify({'content': response.text})
    except Exception as e:
        return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    app.run(debug=True)

```

Secure Code

```

from flask import Flask, request, jsonify
import requests
from urllib.parse import urlparse

app = Flask(__name__)

# Define a whitelist of allowed domains
ALLOWED_DOMAINS = ['example.com', 'api.example.com']

def is_url_allowed(url):
    parsed_url = urlparse(url)
    if parsed_url.scheme not in ['http', 'https']:
        return False
    if parsed_url.hostname not in ALLOWED_DOMAINS:
        return False
    return True

@app.route('/post', methods=['POST'])
def post_fetch():
    url = request.json.get('url')
    data = request.json.get('data', {})

    if not url:
        return jsonify({'error': 'No URL provided'}), 400

    if not is_url_allowed(url):
        return jsonify({'error': 'URL is not allowed'}), 403

    try:
        response = requests.post(url, data=data, allow_redirects=False)
        return jsonify({'content': response.text})
    except Exception as e:
        return jsonify({'error': str(e)}), 500

if __name__ == '__main__':
    app.run(debug=True)

```

API4:2023 - Unrestricted Resource Consumption

Ranking Reason

This can lead to Denial of Service (DoS) attacks, disrupting service availability and affecting user experience.

Explanation

Unrestricted Resource Consumption occurs when an application does not properly limit resource usage, leading to potential Denial of Service (DoS) attacks.

Examples

Example 1: Unrestricted Loop in a Route

In this example, the application accepts a user input to control the number of iterations in a loop, which can lead to high CPU usage.

Vulnerable Code

```
from flask import Flask, request

app = Flask(__name__)

@app.route('/compute')
def compute():
    iterations = int(request.args.get('iterations', '1000')) # default to 1000 if not
    provided
    result = 0
    for i in range(iterations):
        result += i * i
    return f"Computed sum of squares: {result}"

if __name__ == '__main__':
    app.run(debug=True)
```

Secure Code

```
from flask import Flask, request, abort

app = Flask(__name__)

@app.route('/compute')
def compute():
    try:
        # Get the iterations parameter and default to 1000 if not provided
        iterations = int(request.args.get('iterations', '1000'))
    except ValueError:
        return "Invalid input: iterations must be an integer", 400

    # Limit the maximum number of iterations to 10,000
    max_iterations = 10000
    if iterations < 1 or iterations > max_iterations:
        return f"Iterations must be between 1 and {max_iterations}", 400

    result = 0
    for i in range(iterations):
```

```

        result += i * i

    return f"Computed sum of squares: {result}"

if __name__ == '__main__':
    app.run(debug=True)

```

Example 2: Unrestricted File Upload

In this example, the application does not impose any restrictions on the size of uploaded files, which can lead to excessive disk space usage.

Vulnerable Code

```

from flask import Flask, request

app = Flask(__name__)

@app.route('/upload', methods=['POST'])
def upload_file():
    if 'file' not in request.files:
        return "No file part"
    file = request.files['file']
    if file.filename == '':
        return "No selected file"
    file.save(f"./uploads/{file.filename}")
    return "File uploaded successfully"

if __name__ == '__main__':
    app.run(debug=True)

```

Secure Code

```

from flask import Flask, request, abort

app = Flask(__name__)

# Set maximum upload size to 16MB (16 * 1024 * 1024 bytes)
app.config['MAX_CONTENT_LENGTH'] = 16 * 1024 * 1024

@app.route('/upload', methods=['POST'])
def upload_file():
    if 'file' not in request.files:
        return "No file part", 400
    file = request.files['file']
    if file.filename == '':
        return "No selected file", 400
    file.save(f"./uploads/{file.filename}")
    return "File uploaded successfully", 200

@app.errorhandler(413)
def request_entity_too_large(error):
    return "File is too large", 413

if __name__ == '__main__':
    app.run(debug=True)

```

Example 3: Unrestricted Database Queries

In this example, the application allows users to specify the number of database records to fetch without any limits, which can lead to high memory usage and slow performance.

Vulnerable Code

```
from flask import Flask, request
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///example.db'
db = SQLAlchemy(app)

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(80), nullable=False)

@app.route('/users')
def get_users():
    limit = int(request.args.get('limit', '100')) # default to 100 if not provided
    users = User.query.limit(limit).all()
    return {"users": [user.name for user in users]}

if __name__ == '__main__':
    app.run(debug=True)
```

Secure Code

```
from flask import Flask, request, jsonify
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///example.db'
db = SQLAlchemy(app)

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(80), nullable=False)

@app.route('/users')
def get_users():
    try:
        limit = int(request.args.get('limit', '100')) # default to 100 if not provided
        MAX_LIMIT = 1000 # define a maximum limit
        if limit > MAX_LIMIT:
            limit = MAX_LIMIT
        users = User.query.limit(limit).all()
        return jsonify({"users": [user.name for user in users]})
    except ValueError:
        return jsonify({"error": "Invalid limit value"}), 400

if __name__ == '__main__':
    app.run(debug=True)
```

API8:2023 - Security Misconfiguration

Ranking Reason

Misconfigurations can lead to various vulnerabilities being exposed, making it easier for attackers to exploit the system.

Explanation

Security Misconfiguration occurs when security settings are not properly configured or left in an insecure state.

Examples

Example 1: Debug Mode Enabled in Production

Enabling debug mode in production can expose detailed error messages and stack traces, which can be exploited by attackers.

Vulnerable Code

```
from flask import Flask

app = Flask(__name__)
app.config['DEBUG'] = True # Debug mode should not be enabled in production

@app.route('/')
def home():
    return "Welcome to the home page!"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Secure Code

```
from flask import Flask
import os

app = Flask(__name__)

# Set the debug mode based on the FLASK_ENV environment variable
if os.getenv('FLASK_ENV') == 'development':
    app.config['DEBUG'] = True
else:
    app.config['DEBUG'] = False

@app.route('/')
def home():
    return "Welcome to the home page!"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Example 2: Using Default Secret Key

Using a default or hard-coded secret key can make the application vulnerable to various attacks, such as session hijacking.

Vulnerable Code

```
from flask import Flask, session

app = Flask(__name__)
app.secret_key = 'defaultsecret' # This should be a strong, random value

@app.route('/')
def home():
    session['user'] = 'admin'
    return "Welcome to the home page!"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Secure Code

```
from flask import Flask, session
import secrets

app = Flask(__name__)
app.secret_key = secrets.token_hex(16) # Generate a strong, random secret key

@app.route('/')
def home():
    session['user'] = 'admin'
    return "Welcome to the home page!"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Example 3: Insecure HTTP Headers

Not configuring secure HTTP headers can expose the application to various attacks, such as clickjacking, MIME type sniffing, etc.

Vulnerable Code

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    response = app.make_response("Welcome to the home page!")
    # Missing security headers
    # response.headers['X-Frame-Options'] = 'DENY'
    # response.headers['X-Content-Type-Options'] = 'nosniff'
    # response.headers['Content-Security-Policy'] = "default-src 'self'"
    return response

if __name__ == '__main__':
```

```
app.run(host='0.0.0.0', port=5000)
```

Secure Code

```
from flask import Flask

app = Flask(__name__)

@app.after_request
def apply_security_headers(response):
    response.headers['X-Frame-Options'] = 'DENY'
    response.headers['X-Content-Type-Options'] = 'nosniff'
    response.headers['Content-Security-Policy'] = "default-src 'self'"
    return response

@app.route('/')
def home():
    return "Welcome to the home page!"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

API3:2023 - Broken Object Property Level Authorization

Ranking Reason

While serious, it typically has a more limited impact compared to object-level authorization issues.

Explanation

Broken Object Property Level Authorization occurs when the application fails to enforce proper access control on object properties, allowing unauthorized modifications.

Examples

Example 1: Lack of Authorization Check

There is no authorization check to verify if the current user is allowed to access the requested user's information. This allows any authenticated user to access any user's details, potentially exposing sensitive information.

Vulnerable Code

```
from flask import Flask, request, jsonify
app = Flask(__name__)

# Dummy data (normally would come from a database)
users = [
    {'id': 1, 'username': 'alice', 'role': 'admin'},
    {'id': 2, 'username': 'bob', 'role': 'user'}
]

@app.route('/api/users/<int:user_id>', methods=['GET'])
def get_user(user_id):
    # Assume user_id comes from the request
    user = next((user for user in users if user['id'] == user_id), None)
```

```

    if not user:
        return jsonify({'error': 'User not found'}), 404

    # No authorization check here!
    return jsonify({'username': user['username']}), 200

if __name__ == '__main__':
    app.run(debug=True)

```

Secure Code

```

from flask import Flask, request, jsonify

app = Flask(__name__)

# Dummy data (normally would come from a database)
users = [
    {'id': 1, 'username': 'alice', 'role': 'admin'},
    {'id': 2, 'username': 'bob', 'role': 'user'}
]

# Simulate current user (for example purposes only)
def get_current_user():
    # In a real application, this should be obtained from session/authentication
    # For example, assume the current user is 'bob' with id 2
    return {'id': 2, 'username': 'bob', 'role': 'user'}

@app.route('/api/users/<int:user_id>', methods=['GET'])
def get_user(user_id):
    # Retrieve the user from the data
    user = next((user for user in users if user['id'] == user_id), None)

    if not user:
        return jsonify({'error': 'User not found'}), 404

    # Get the current user
    current_user = get_current_user()

    # Authorization check
    if current_user['role'] != 'admin' and current_user['id'] != user_id:
        return jsonify({'error': 'Unauthorized access'}), 403

    return jsonify({'username': user['username']}), 200

if __name__ == '__main__':
    app.run(debug=True)

```

Example 2: Insecure Direct Object Reference (IDOR)

The authorization check is insufficient, as it only verifies if the current user has an 'admin' role, ignoring other potential authorization criteria (e.g., user-specific permissions). This can lead to unauthorized access if roles are not properly managed or if an attacker manipulates the request parameters.

Vulnerable Code

```
from flask import Flask, request, jsonify
app = Flask(__name__)

# Dummy data (normally would come from a database)
users = [
    {'id': 1, 'username': 'alice', 'role': 'admin'},
    {'id': 2, 'username': 'bob', 'role': 'user'}
]

@app.route('/api/users/<int:user_id>', methods=['GET'])
def get_user(user_id):
    # Assume user_id comes from the request
    user = next((user for user in users if user['id'] == user_id), None)

    if not user:
        return jsonify({'error': 'User not found'}), 404

    # Incorrect authorization check: checking for admin role only
    if request.args.get('role') != 'admin':
        return jsonify({'error': 'Unauthorized access'}), 403

    return jsonify({'username': user['username']}), 200

if __name__ == '__main__':
    app.run(debug=True)
```

Secure Code

```
from flask import Flask, request, jsonify
app = Flask(__name__)

# Dummy data (normally would come from a database)
users = [
    {'id': 1, 'username': 'alice', 'role': 'admin'},
    {'id': 2, 'username': 'bob', 'role': 'user'}
]

# Dummy current user for demonstration
# In a real application, you would get this from session or authentication token
current_user = {'id': 1, 'username': 'alice', 'role': 'admin'}

@app.route('/api/users/<int:user_id>', methods=['GET'])
def get_user(user_id):
    # Assume user_id comes from the request
    user = next((user for user in users if user['id'] == user_id), None)

    if not user:
        return jsonify({'error': 'User not found'}), 404

    # Authorization check: verify if the current user has permission
    if current_user['role'] != 'admin' and current_user['id'] != user_id:
        return jsonify({'error': 'Unauthorized access'}), 403
```

```

        return jsonify({'username': user['username']}), 200

if __name__ == '__main__':
    app.run(debug=True)

```

Example 3: Improper Use of Object References

This example shows how assuming properties about users without proper authorization checks can lead to vulnerabilities. In this case, the code assumes that if the user's username is 'alice', access should be denied, but this is not a robust or scalable method for authorization.

Vulnerable Code

```

from flask import Flask, request, jsonify
app = Flask(__name__)

# Dummy data (normally would come from a database)
users = [
    {'id': 1, 'username': 'alice', 'role': 'admin'},
    {'id': 2, 'username': 'bob', 'role': 'user'}
]

@app.route('/api/users/<int:user_id>', methods=['GET'])
def get_user(user_id):
    # Assume user_id comes from the request
    user = next((user for user in users if user['id'] == user_id), None)

    if not user:
        return jsonify({'error': 'User not found'}), 404

    # Simulating improper use of user data based on an assumption
    if user['username'] == 'alice':
        return jsonify({'error': 'Unauthorized access to Alice\'s account'}), 403

    return jsonify({'username': user['username']}), 200

if __name__ == '__main__':
    app.run(debug=True)

```

Secure Code

```

from flask import Flask, request, jsonify
app = Flask(__name__)

# Dummy data (normally would come from a database)
users = [
    {'id': 1, 'username': 'alice', 'role': 'admin'},
    {'id': 2, 'username': 'bob', 'role': 'user'}
]

# Mock authentication function to simulate a logged-in user
# In a real application, this should come from a session or token
def get_logged_in_user():
    # Simulate a logged-in user with role 'user' (could be 'admin' for different
    access)
    return {'id': 2, 'username': 'bob', 'role': 'user'}

```

```

@app.route('/api/users/<int:user_id>', methods=['GET'])
def get_user(user_id):
    user = next((user for user in users if user['id'] == user_id), None)

    if not user:
        return jsonify({'error': 'User not found'}), 404

    logged_in_user = get_logged_in_user()

    # Example of role-based access control
    if logged_in_user['role'] != 'admin' and logged_in_user['id'] != user_id:
        return jsonify({'error': 'Unauthorized access'}), 403

    return jsonify({'username': user['username']}), 200

if __name__ == '__main__':
    app.run(debug=True)

```

API10:2023 - Unsafe Consumption of APIs

Ranking Reason

Consuming untrusted APIs can lead to vulnerabilities, but it usually requires additional factors to be exploited effectively.

Explanation

Unsafe Consumption of APIs occurs when an application consumes APIs in an insecure manner, leading to potential security vulnerabilities.

Examples

Example 1: Lack of Input Validation

This code does not validate or sanitize the `user_input` parameter before using it to query an external API. An attacker could manipulate `user_input` to inject malicious input or perform unauthorized actions through the external API.

Vulnerable Code

```

from flask import Flask, request, jsonify
import requests

app = Flask(__name__)

@app.route('/vulnerable_api', methods=['POST'])
def vulnerable_api():
    user_input = request.json.get('user_input')

    # Assume this endpoint consumes an external API
    response = requests.get(f'https://external.api.com/data/{user_input}')

    return jsonify(response.json()), 200

```

Secure Code

```
from flask import Flask, request, jsonify, abort
import requests
import re

app = Flask(__name__)

@app.route('/vulnerable_api', methods=['POST'])
def vulnerable_api():
    user_input = request.json.get('user_input')

    # Validate input
    if not user_input:
        abort(400, description="user_input is required")

    # Example validation: Ensure user_input is a valid alphanumeric string
    if not isinstance(user_input, str) or not re.match(r'^[a-zA-Z0-9_-]+$',
user_input):
        abort(400, description="Invalid user_input format")

    # Sanitize input
    sanitized_input = re.sub(r'^[a-zA-Z0-9_-]', '', user_input)

    # Use parameterized URL construction to avoid injection issues
    url = f'https://external.api.com/data/{sanitized_input}'
    response = requests.get(url)

    # Handle response errors
    if response.status_code != 200:
        abort(response.status_code, description="External API request failed")

    return jsonify(response.json()), 200
```

Example 2: Insufficient Authentication Check

While this code checks for an API key, it does not validate the contents of `user_input`. An attacker could exploit this by bypassing the API key check or injecting malicious input into `user_input`.

Vulnerable Code

```
from flask import Flask, request, jsonify
import requests

app = Flask(__name__)

@app.route('/secure_api', methods=['POST'])
def secure_api():
    api_key = request.headers.get('X-API-Key')
    if not api_key:
        return jsonify({'error': 'Unauthorized access'}), 401

    user_input = request.json.get('user_input')

    # Assume this endpoint consumes an authenticated external API
    response = requests.post(
```

```

        'https://external.api.com/data',
        headers={'X-API-Key': api_key},
        json={'user_input': user_input}
    )

    return jsonify(response.json()), 200

```

Secure Code

```

from flask import Flask, request, jsonify
import requests
import re

app = Flask(__name__)

# Define a function to validate the API key
def validate_api_key(api_key):
    # Implement your API key validation logic here
    # For example, you might check against a list of valid keys or a database
    return api_key == 'your-valid-api-key'

# Define a function to sanitize user input
def sanitize_input(user_input):
    # Perform input sanitization to prevent injection attacks
    if not isinstance(user_input, str):
        raise ValueError("Invalid input type")
    sanitized_input = re.sub(r'[\W\s]', '', user_input) # Remove non-alphanumeric
    characters
    return sanitized_input

@app.route('/secure_api', methods=['POST'])
def secure_api():
    api_key = request.headers.get('X-API-Key')
    if not api_key or not validate_api_key(api_key):
        return jsonify({'error': 'Unauthorized access'}), 401

    user_input = request.json.get('user_input')
    if user_input is None:
        return jsonify({'error': 'No input provided'}), 400

    try:
        sanitized_input = sanitize_input(user_input)
    except ValueError as e:
        return jsonify({'error': str(e)}), 400

    # Assume this endpoint consumes an authenticated external API
    try:
        response = requests.post(
            'https://external.api.com/data',
            headers={'X-API-Key': api_key},
            json={'user_input': sanitized_input}
        )
        response.raise_for_status() # Raise an exception for HTTP errors
    except requests.RequestException as e:
        return jsonify({'error': 'External API request failed', 'details': str(e)}),
500

```

```
return jsonify(response.json()), 200
```

Example 3: Insecure Data Transmission

This code sends `user_input` to an external API over an insecure HTTP connection (`http://`). This exposes the data to potential interception and modification by attackers, compromising its confidentiality and integrity.

Vulnerable Code

```
from flask import Flask, request, jsonify
import requests

app = Flask(__name__)

@app.route('/transmit_data', methods=['POST'])
def transmit_data():
    user_input = request.json.get('user_input')

    # Assume this endpoint transmits user input to an external API insecurely
    response = requests.post(
        'http://insecure.api.com/transmit',
        json={'data': user_input}
    )

    return jsonify(response.json()), 200
```

Secure Code

```
from flask import Flask, request, jsonify
import requests

app = Flask(__name__)

@app.route('/transmit_data', methods=['POST'])
def transmit_data():
    user_input = request.json.get('user_input')

    # Updated endpoint to use HTTPS for secure communication
    response = requests.post(
        'https://secure.api.com/transmit', # Use HTTPS for secure connection
        json={'data': user_input}
    )

    return jsonify(response.json()), 200
```

API9:2023 - Improper Inventory Management

Ranking Reason

While important for security hygiene, it usually does not directly lead to critical vulnerabilities without other contributing factors.

Explanation

Improper Inventory Management occurs when APIs are not properly documented or maintained, leading to outdated or insecure endpoints being exposed.

Examples

Example 1: Lack of Input Validation

This code lacks proper input validation. An attacker could exploit this by sending negative values for `quantity`, causing the inventory count to decrease unexpectedly.

Vulnerable Code

```
from flask import Flask, request, jsonify

app = Flask(__name__)

inventory = {
    'item1': 100,
    'item2': 50,
    'item3': 75
}

@app.route('/update_inventory', methods=['POST'])
def update_inventory():
    data = request.get_json()

    item_name = data.get('item_name')
    quantity = data.get('quantity')

    if item_name and quantity:
        if item_name in inventory:
            inventory[item_name] += quantity # Vulnerable code: No input validation
            return jsonify({'message': 'Inventory updated successfully'}), 200
        else:
            return jsonify({'error': 'Item not found in inventory'}), 404
    else:
        return jsonify({'error': 'Invalid input data'}), 400

if __name__ == '__main__':
    app.run(debug=True)
```

Secure Code

```
from flask import Flask, request, jsonify

app = Flask(__name__)

inventory = {
    'item1': 100,
    'item2': 50,
    'item3': 75
}

@app.route('/update_inventory', methods=['POST'])
def update_inventory():
```

```

data = request.get_json()

item_name = data.get('item_name')
quantity = data.get('quantity')

if item_name and quantity is not None:
    # Validate that quantity is a non-negative integer
    if not isinstance(quantity, int) or quantity < 0:
        return jsonify({'error': 'Invalid quantity. It must be a non-negative integer.'}), 400

    if item_name in inventory:
        inventory[item_name] += quantity
        return jsonify({'message': 'Inventory updated successfully'}), 200
    else:
        return jsonify({'error': 'Item not found in inventory'}), 404
else:
    return jsonify({'error': 'Invalid input data'}), 400

if __name__ == '__main__':
    app.run(debug=True)

```

Example 2: Insecure Direct Object Reference (IDOR)

This endpoint allows direct access to inventory items based on their names (`item_name`). An attacker could manipulate the `item_name` parameter to access items they should not have permissions to view, potentially exposing sensitive inventory details.

Vulnerable Code

```

from flask import Flask, request, jsonify

app = Flask(__name__)

inventory = {
    'item1': 100,
    'item2': 50,
    'item3': 75
}

@app.route('/get_inventory/<item_name>', methods=['GET'])
def get_inventory(item_name):
    if item_name in inventory:
        return jsonify({'item_name': item_name, 'quantity': inventory[item_name]}), 200
    else:
        return jsonify({'error': 'Item not found in inventory'}), 404

if __name__ == '__main__':
    app.run(debug=True)

```

Secure Code

```

from flask import Flask, request, jsonify, abort
from functools import wraps

app = Flask(__name__)

```



```

# Sample inventory
inventory = {
    'item1': 100,
    'item2': 50,
    'item3': 75
}

# Basic authentication credentials (username:password)
AUTHORIZED_USERS = {
    'user1': 'password123'
}

def check_auth(username, password):
    return AUTHORIZED_USERS.get(username) == password

def authenticate():
    return jsonify({'error': 'Authentication required'}), 401

def requires_auth(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        auth = request.authorization
        if not auth or not check_auth(auth.username, auth.password):
            return authenticate()
        return f(*args, **kwargs)
    return decorated

@app.route('/get_inventory/<item_name>', methods=['GET'])
@requires_auth
def get_inventory(item_name):
    # Basic validation of item_name
    if not item_name.isalnum() or len(item_name) > 50:
        return jsonify({'error': 'Invalid item name'}), 400

    if item_name in inventory:
        return jsonify({'item_name': item_name, 'quantity': inventory[item_name]}), 200
    else:
        return jsonify({'error': 'Item not found in inventory'}), 404

if __name__ == '__main__':
    app.run(debug=True)

```

Example 3: Insufficient Authentication and Authorization

This code relies solely on a simple token-based authentication mechanism (`admin_token`). There are no checks for user roles or permissions beyond the token itself, potentially allowing unauthorized users with the token to modify the inventory.

Vulnerable Code

```

from flask import Flask, request, jsonify

app = Flask(__name__)

admin_token = 'secret_admin_token'

```

```

inventory = {
    'item1': 100,
    'item2': 50,
    'item3': 75
}

@app.route('/add_item', methods=['POST'])
def add_item():
    token = request.headers.get('Authorization')

    if token == admin_token:
        data = request.get_json()
        item_name = data.get('item_name')
        quantity = data.get('quantity')

        if item_name and quantity:
            inventory[item_name] = quantity # Vulnerable code: No further
authorization checks
            return jsonify({'message': 'Item added to inventory'}), 200
        else:
            return jsonify({'error': 'Invalid input data'}), 400
    else:
        return jsonify({'error': 'Unauthorized access'}), 401

if __name__ == '__main__':
    app.run(debug=True)

```

Secure Code

```

from flask import Flask, request, jsonify
from functools import wraps
import jwt
import datetime

app = Flask(__name__)

# Secret key for JWT encoding/decoding
SECRET_KEY = 'your_secret_key'

# Example of a token for an admin (for demonstration purposes)
admin_token = jwt.encode({'role': 'admin', 'exp': datetime.datetime.utcnow() +
datetime.timedelta(hours=1)}, SECRET_KEY, algorithm='HS256')

inventory = {
    'item1': 100,
    'item2': 50,
    'item3': 75
}

def token_required(f):
    @wraps(f)
    def decorated_function(*args, **kwargs):
        token = request.headers.get('Authorization')
        if not token:
            return jsonify({'error': 'Token is missing'}), 401

```

```
try:
    data = jwt.decode(token, SECRET_KEY, algorithms=['HS256'])
    if data['role'] != 'admin':
        return jsonify({'error': 'Permission denied'}), 403
except jwt.ExpiredSignatureError:
    return jsonify({'error': 'Token has expired'}), 401
except jwt.InvalidTokenError:
    return jsonify({'error': 'Invalid token'}), 401
return f(*args, **kwargs)
return decorated_function

@app.route('/add_item', methods=['POST'])
@token_required
def add_item():
    data = request.get_json()
    item_name = data.get('item_name')
    quantity = data.get('quantity')

    if item_name and quantity:
        inventory[item_name] = quantity
        return jsonify({'message': 'Item added to inventory'}), 200
    else:
        return jsonify({'error': 'Invalid input data'}), 400

if __name__ == '__main__':
    app.run(debug=True)
```

Conclusion

These examples demonstrate common vulnerabilities and how they can manifest in code and their fixes. Always ensure to follow best practices for secure coding, such as proper authentication, authorization, input validation, and secure configuration to avoid these pitfalls.
